

Joc PAOO

Botez Ilie și Sava Teofan

Grupa: 1213A

Povestea jocului:

Acțiunea jocului se petrece în Evul Mediu, unde Arthur este un cavaler al vremii. Niște monștri, conduși de Regele Orcilor îi răpesc soția și o duc în Castelul Întunecat, unde o închide în turn. Arthur se duce și va face tot posibilul să o salveze pe soția lui, folosindu-și abilitățile de luptă și dexteritatea pentru a trece de camerele castelului și pentru a ajunge în timp la soție.

Niveluri:

Nivelul 1: Acțiunea se petrece într-o temniță. Luminozitatea este scăzută, poți vedea doar în fața personajului. Trebuie să găsești o cheie pentru a putea deschide o ușă și a trece la nivelul următor. Arthur va avea o sabie cu care va ataca inamicii (orci).

Nivelul 2: Personajul ajunge într-o cameră mare, cuprinsă de gheață și plină de capcane la tot pasul. Podeaua este înghețată și Arthur va îngheța dacă stă prea mult pe loc și va aluneca când se mișcă.

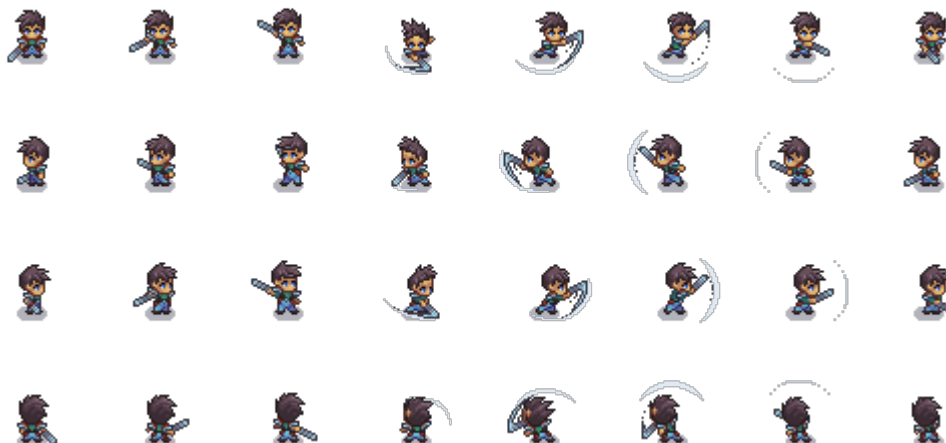
Nivelul 3: Nivelul final, personajul principal să se lupte cu Regele Orcilor, care crează monștri. Acesta va crea slime-uri care să îl atace pe Arthur.

Harta: Toate hărțile (create vizual în Tiled și exportate în format .json) sunt încărcate în memorie de la pornirea jocului, într-un vector `maps[]`. Efectul de adâncime 2.5D este obținut prin randarea hărții pe straturi (layer) suprapuse, permițând jucătorului să se deplaseze pe podea, dar și să se ascundă sub obiecte sau după ziduri.

Camera: Clasa Camera este utilizată pentru a aplica un zoom asupra hărții, asigurând vizibilitatea secțiunii corecte a zonei în care se află personajul.

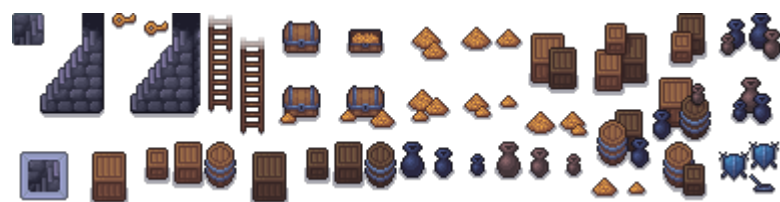
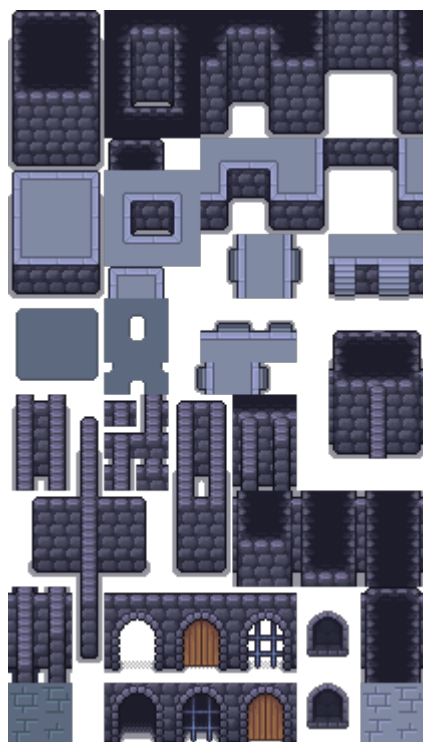
Sprite sheet-uri:

Personajul principal:

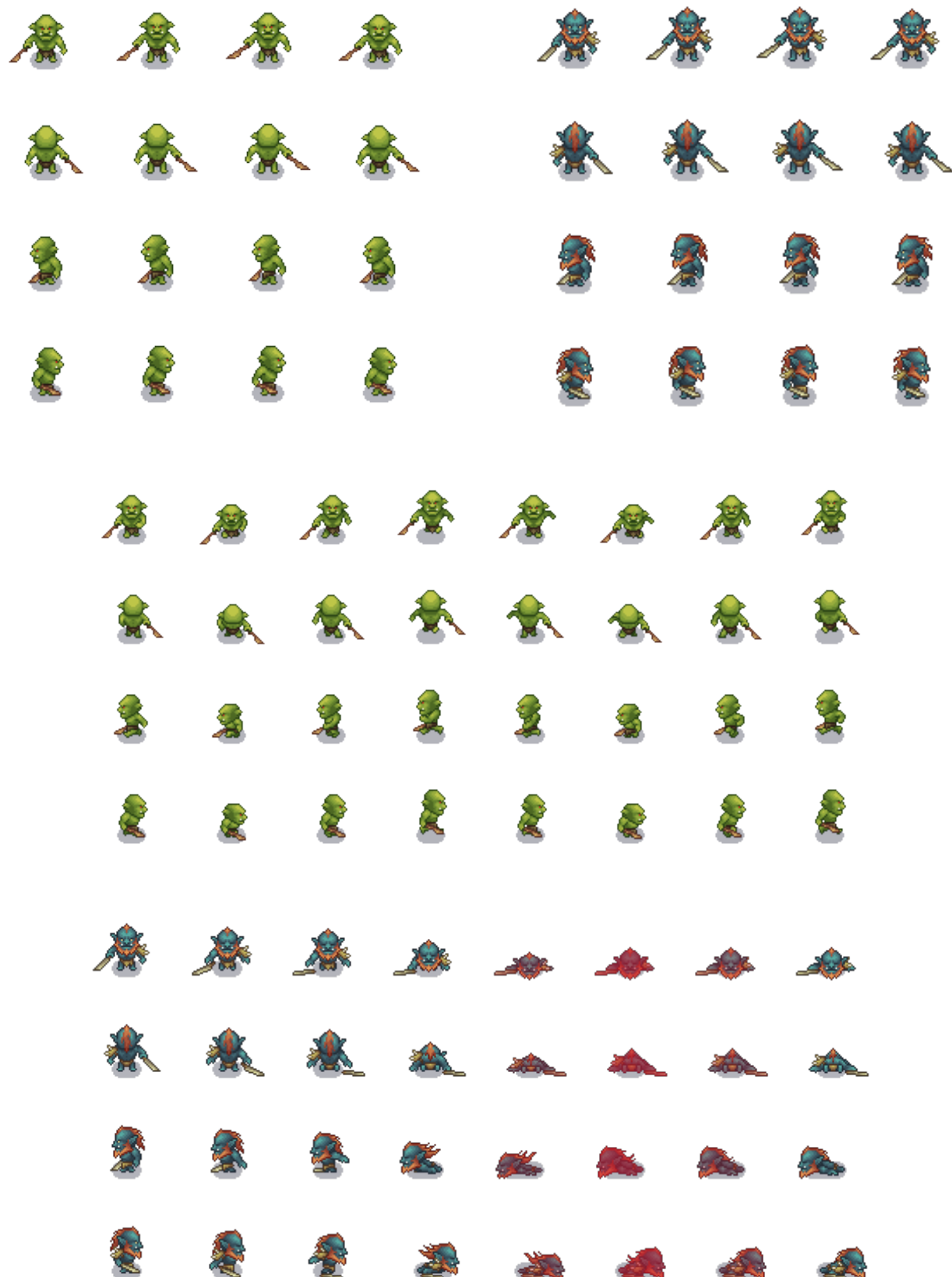




Harta pentru nivelul 1:



Orcii:



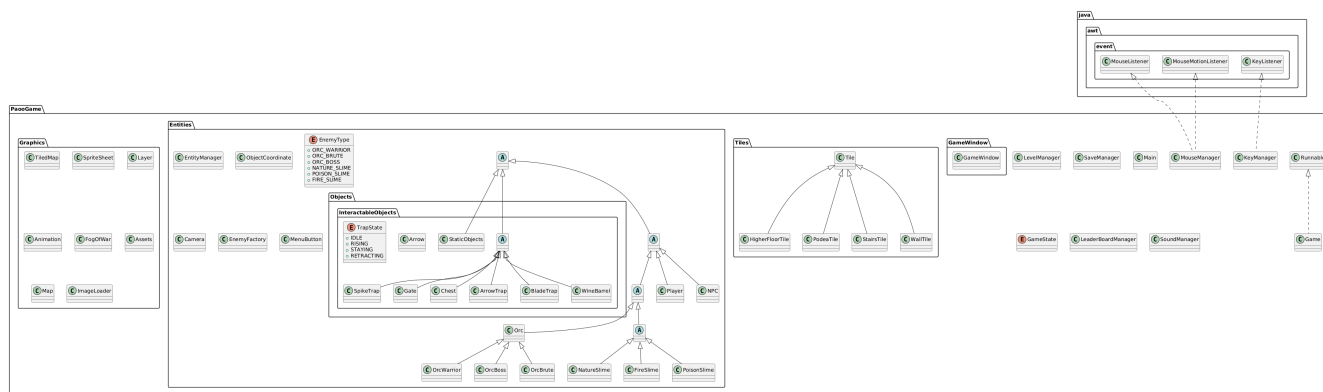
Gameplay:

Jocul este structurat sub forma unui sistem software care rulează într-o buclă infinită, actualizând starea lumii de 60 de ori pe secundă. Obiectivul central al jucătorului este explorarea hărții și parcurgerea a trei niveluri distincte pentru a finaliza misiunile personajului principal. Jucătorul trebuie să ia decizii pe moment pentru a atinge scopul fiecărui nivel, respectând setul de reguli impus de mediu.

Personajul principal posedă atribute precum viață (Health) și viteză de deplasare (walkSpeed sau runSpeed). Deplasarea se realizează prin modificarea coordonatelor pe axele X și Y în funcție de input-ul de la tastatură. Interacțiunea cu mediul se realizează prin intermediul unui sistem de coliziuni. Anumite elemente de decor (ex: perete, apă) sunt marcate ca fiind de tip Solid, generând „mutări imposibile” care blochează trecerea personajului. Jucătorul pierde în momentul în care viața personajului ajunge la zero, prin coliziuni repetate cu inamicii sau capcanele.

Caracterului merge la interacțiunea jucătorului cu tastele Up, Left, Down, Right sau W, A, S, D. Pentru a face personajul să alerge se ține apăsată tasta Shift Left sau Shift Right, iar pentru a ataca se apasă tasta Space. Tasta E este folosită pentru a interacționa cu anumite obiecte din mediul înconjurător.

Diagrama UML:



Sabloane de proiectare:

1. Șablonul "Main Loop": Acest șablon reprezintă nucleul central al oricărei aplicații grafice interactive, asigurând rularea continuă a programului într-o buclă infinită. Utilizarea acestui model în clasa Game este vitală pentru a decupla progresia timpului din joc de viteza de execuție a procesorului. Prin implementarea metodei run(), aplicația garantează o rată de înprospătare constantă de 60 FPS, procesând succesiv input-ul utilizatorului, actualizarea stărilor interne (Update()) și randarea cadrelor pe ecran (Draw()) fără a bloca execuția. Această abordare permite o animație fluentă și o interacțiune predictibilă, indiferent de performanța hardware a sistemului.

2. Șablonul Flyweight: Șablonul Flyweight este o tehnică de optimizare structurală care vizează reducerea consumului de memorie prin partajarea obiectelor care conțin date comune. În dezvoltarea hărților bazate pe dale (tilemaps), încărcarea repetată a aceleiași texturi pentru fiecare celulă ar genera un consum inutil de resurse. În proiect, acest șablon este implementat prin ierarhia clasei Tile. Toate

tipurile de dale sunt instanțiate o singură dată și stocate într-un vector static de referințe în clasa Tile. În timpul randării hărții, programul nu creează obiecte noi, ci transmite doar referința obiectului preexistent și coordonatele specifice, asigurând o încărcare minimă a memoriei și o performanță ridicată a sistemului de afișare.

3. Șablonul Factory: Acest șablon reprezintă un model de proiectare care oferă o interfață generică pentru instanțierea obiectelor, delegând responsabilitatea creării subclaselor specifice. În cadrul proiectului, utilitatea acestui model este evidențiată prin clasa EnemyFactory, care centralizează logica de generare a diverselor tipuri de inamici (precum Orc1 sau Orc2). Prin implementarea acestui șablon, se realizează o separare între logica de gestionare a nivelurilor (LevelManager) și clasele concrete ale entităților, simplificând structura programului. Această abordare permite o ușoară extindere a jocului pe viitor și permite adăugarea unor inamici noi fără a schimba codul existent.

User Interface Design:

Proiectarea interfeței a vizat crearea unui mediu intuitiv care să nu distragă atenția de la gameplay-ul jocului. Arhitectura UI este compusă dintr-o ierarhie de ecrane gestionate prin obiectele JFrame și Canvas. Meniul Principal utilizează elemente grafice personalizate pentru butoanele de „Start” și „Exit”, încărcate prin clasa Assets.

Interacțiunea personajului cu mediul este gestionată prin KeyManager, oferind feedback vizual la selecție. Interfața de Joc include un sistem de tip „Head-Up Display” (HUD) care afișează dinamic starea de sănătate a entităților. Inamicii dispun de bare de viață randate deasupra sprite-urilor doar atunci când aceștia sunt răniți, optimizând astfel claritatea vizuală.

Elemente de Testare

Stabilitatea cadrelor (FPS) s-a monitorizat constant timpul de execuție al metodelor Update() și Draw() pentru a asigura menținerea standardului de 60 FPS . S-a observat că utilizarea șablonului Flyweight pentru dale a redus semnificativ consumul de memorie în timpul testelor pe hărți de dimensiuni mari.

Testarea coliziunilor s-au efectuat teste de tip „boundary check” pentru a valida proprietatea IsSolid a pereților. O atenție deosebită a fost acordată clasei StairsTile, unde s-a verificat prin „hitbox-uri” vizuale (randate cu roșu în modul de debug) dacă marginile de coliziune permit urcarea.

Comportamentul orcilor a fost testat prin simularea distanțelor de „chase” și „attack”. S-a verificat dacă metoda canMoveWithoutStacking() previne corect suprapunerea inamicilor atunci când aceștia urmăresc jucătorul în grup.

Deoarece jocul rulează la o rezoluție ridicată, randarea exclusiv prin software (CPU) poate duce la scăderi de cadre (lag) în momentele cu multe entități pe ecran. De aceea, în clasa Main s-a activat

proprietatea sun.java2d.opengl, forțând mașina virtuală Java să utilizeze unitatea de procesare grafică (GPU) pentru desenarea elementelor. Această optimizare a permis o randare mult mai eficientă a texturilor de mari dimensiuni din Assets și a hărților complexe.

O problemă critică identificată a fost tendința buclei de tip „Main Loop” de a consuma resursele procesorului în mod inutil prin procesul de „busy-waiting”. S-a introdus mecanismul Thread.sleep(1) în cadrul clasei Game. Această „pauză” controlată permite sistemului de operare să aloce resurse și altor procese, asigurându-se totodată că jocul nu rulează mai repede decât pragul setat de 60 FPS. S-a obținut o sincronizare stabilă a cadrelor și prin testele de performanță realizate s-a observat o scădere a gradului de ocupare al procesorului (CPU Usage) de la 13-14% la 2-3%.

Lista sarcinilor săptămânale:

Săptămâna 1:

Botez Ilie – Realizarea unei hărți de test, prin adăugarea dalelor și implementarea coliziunilor cu mediul

Sava Teofan – Încărcarea sprite-sheet-urilor necesare pentru realizarea graficii jocului și crearea personajului principal, integrând comenzile de la tastatură pentru mișcare și atac

Săptămâna 2:

Botez Ilie – Implementarea unei camere care urmărește personajul și dă zoom

Sava Teofan - Implementarea orcilor, restructurarea claselor și crearea arhitecturii de bază pentru entități

Săptămâna 3:

Botez Ilie – Crearea meniului principal și realizarea unei animații de fade-in la schimbarea scenelor

Sava Teofan - Implementarea unui șablon de proiectare tip Factory și optimizarea logicii de luptă și a deplasării orcilor

Săptămâna 4:

Botez Ilie – Implementarea hărții pentru primul nivel cu următoarele mecanici: butoaie cu vin care îți refac din viața, dar îți inversează controalele pentru o perioadă, un cufar cu o cheie pentru a deschide poarta către următorul nivel și fog of war

Sava Teofan – Îmbunătățirea și extinderea meniului principal, crearea și dezvoltarea unui meniu de pauză și a unui meniu de setări, îmbunătățiri la gameplay și adăugarea unei resurse audio

Săptămâna 5:

Botez Ilie – Adăugarea unui cronometru și a unui scor, creare hashmap pentru asset-uri, actualizare clasa de save pentru a salva mai mult, adăugarea decorațiunilor în primul nivel și a unui timer pentru debuff-ul de drunk

Sava Teofan – Implementarea unui buton și a meniului pentru tutorial

Săptămâna 6:

Botez Ilie – Încărcarea hărților pentru nivelul doi și trei

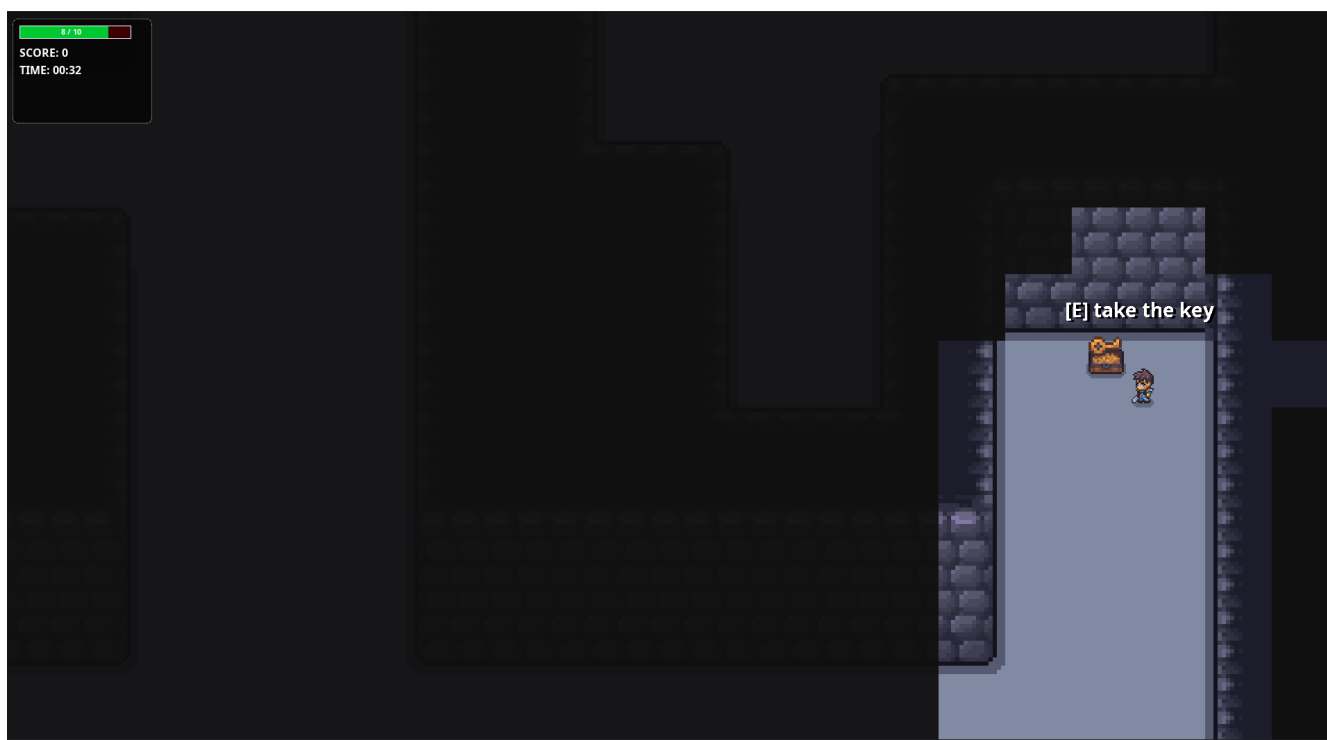
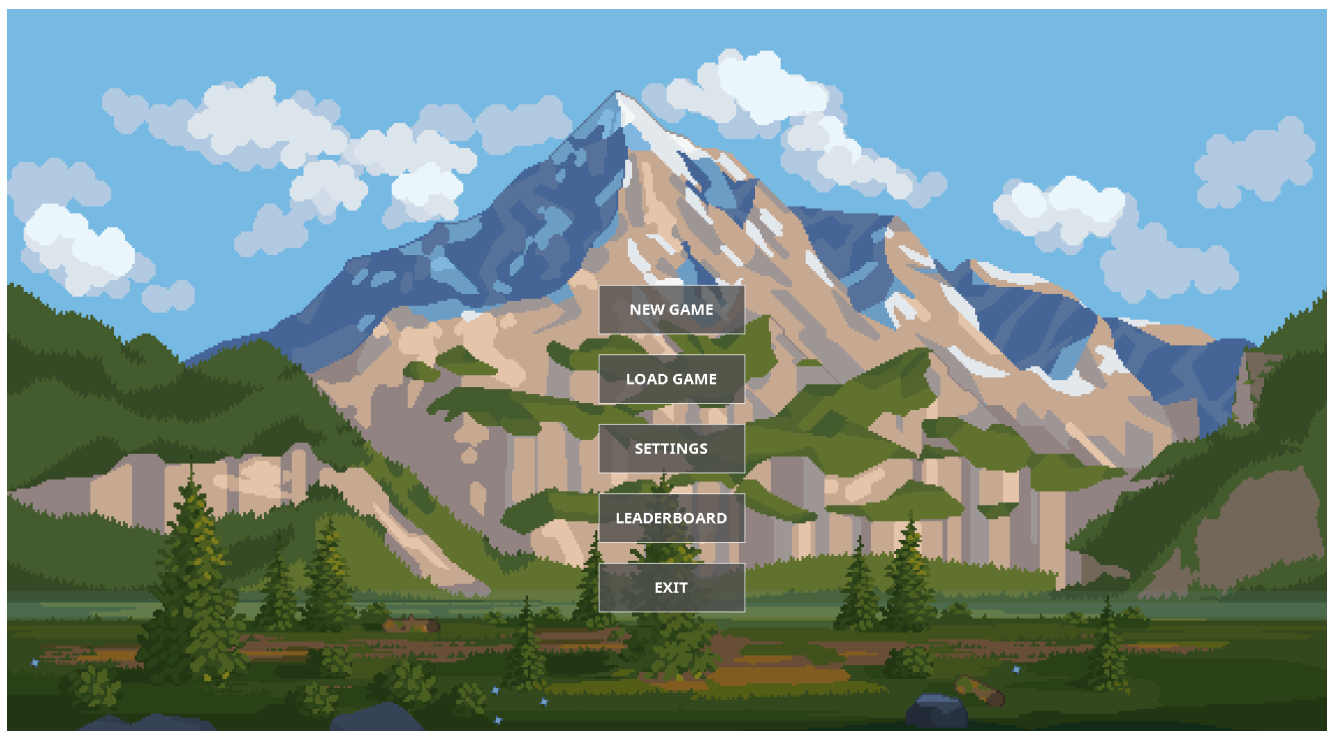
Sava Teofan – Adăugare NPC

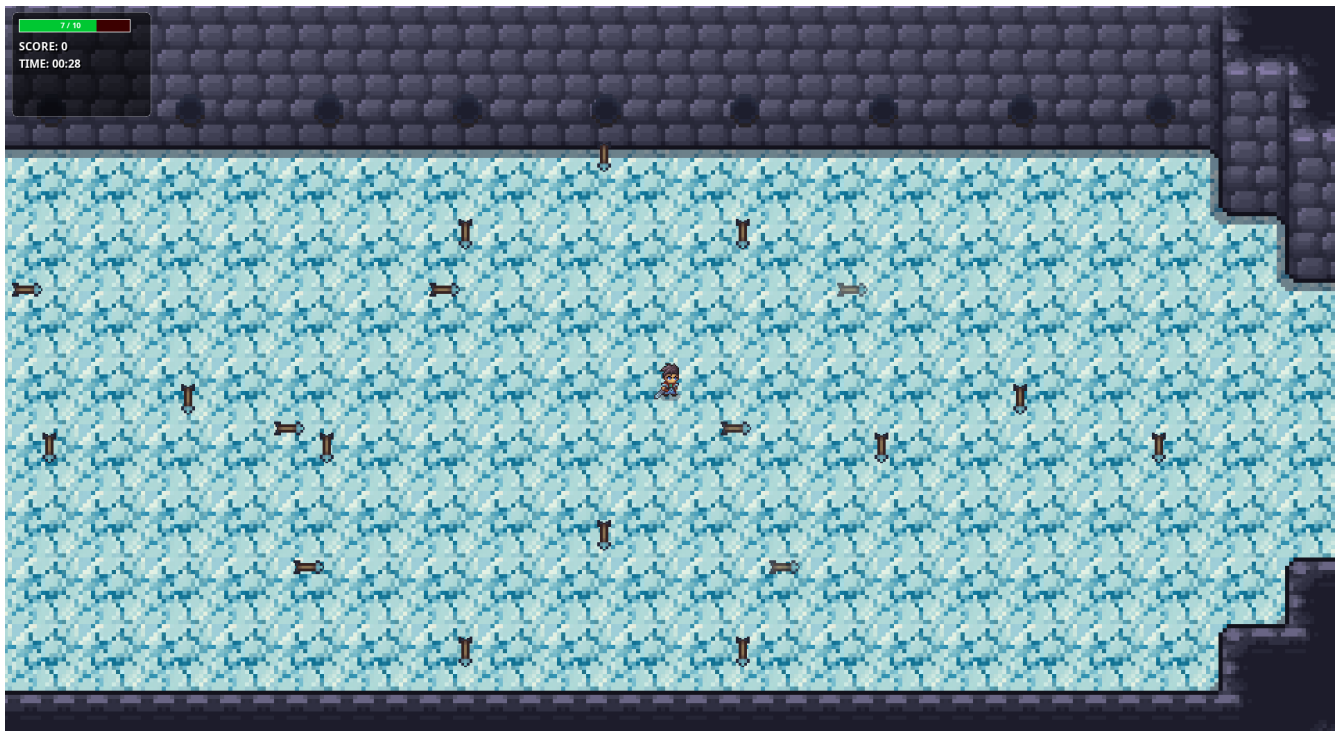
Săptămâna 7:

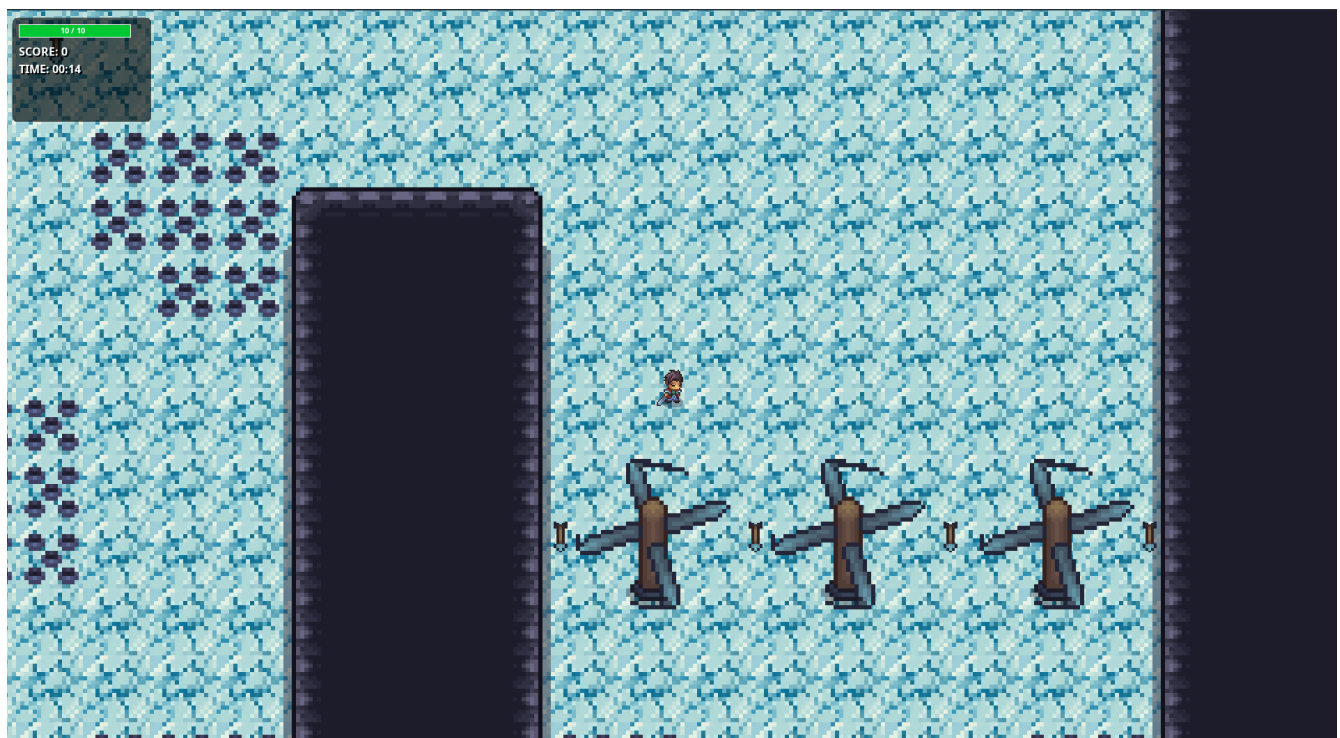
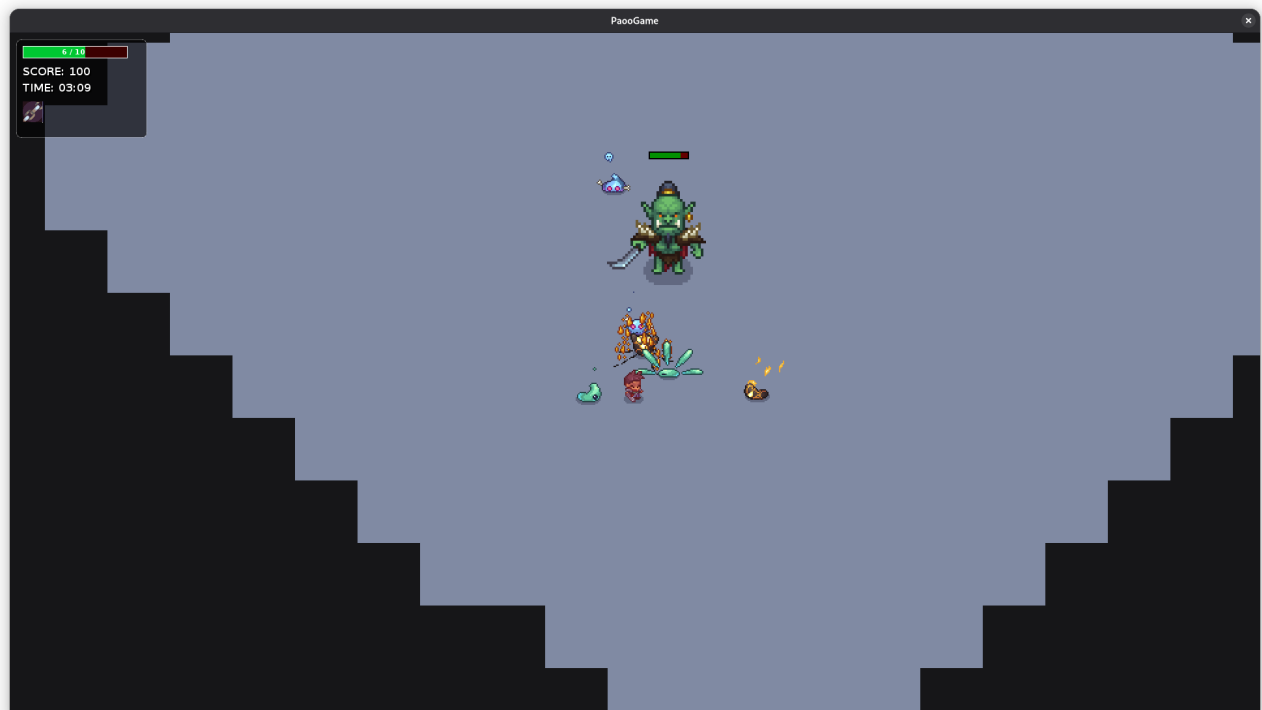
Botez Ilie – Crearea unui leaderboard pentru cei mai buni 10 jucători, cu o bază de date făcută în SQL, pregătirea nivelului 2 și adăugarea capcanelor(săgeți, țepi, lame) și ajustări la cod

Sava Teofan – Implementarea nivelului 3, prin crearea boss-ului și a slime-urilor, adăugarea efectului de gheață la nivelul 2 și ajustări la cod

Screenshot-uri din joc:







Bibliografie:

Sprite-sheet-uri: <https://craftpix.net/>

Java language: <https://www.codecademy.com/learn/learn-java> și <https://academy.oracle.com/>

"Game Programming Patterns: Game Loop", Robert Nystrom.
<https://gameprogrammingpatterns.com/game-loop.html>

"Gson User Guide", GitHub.